Sub-module 4

Memory-Mapped I/O PicoRV32 + UART

How a RISC-V core talks to a UART through memory-mapped I/O
Covers hardware architecture, bus transactions, and firmware

From MMIO concept → bus protocol → register map → C firmware

What We'll Cover

01 What is MMIO?

02 PicoRV32 SoC - Complete Memory Map

03 AXI Decode Address

04 Block Diagram: AXI→Decoder→UART

05 Firmware UART Driver — fw/main.c

06 MMIO Write Path: uart_putc('H') Full Trace

07 Lab: make soc_pico_uart_mem

08 Design Rules

09 Summary

What is Memory-Mapped I/O?

Analogy:

RISC-V has no special IN/OUT instructions. Every peripheral — including UART — sits at a fixed physical address. The CPU uses ordinary lw / sw instructions; the interconnect routes the access to the right device.

What MMIO Defines

- Fixed physical address per peripheral
- Normal load/store = device access
- No special I/O instructions needed
- Address bus decoded by interconnect

Why It Matters

- Multiple peripherals share one bus
- Avoids data corruption & conflicts
- Enables modular SoC design
- Reusable IP blocks with fixed maps

UART in MMIO World

- UART sits at e.g. 0x10013000
- CPU writes txdata → UART transmits
- CPU reads rxdata → UART receives
- Control regs configure baud, stop bits

PicoRV32 SoC - Complete Memory Map

Address Range	Peripheral	Bus	Size
`0x0000_0000–0x0000_FFFF`	**ROM**	AXI-Lite S0	64 KB win / 8 KB deep
`0x0001_0000–0x0001_FFFF`	**SRAM**	AXI-Lite S1	64 KB win / 256 B deep
`0x1000_0000`	**UART TX**	AXI-Lite S2	Write → transmit byte
`0x1000_0004`	**UART RX**	AXI-Lite S2	Read → received byte
`0x1000_0008`	**UART STATUS**	AXI-Lite S2	bit[1] = rx_buf_valid

STATUS Reg — 0x1000_0008

`Bit [0]`	reserved = 0
`Bit [1]`	rx_buf_valid — RX byte ready
`Bits [31:2]`	reserved = 0

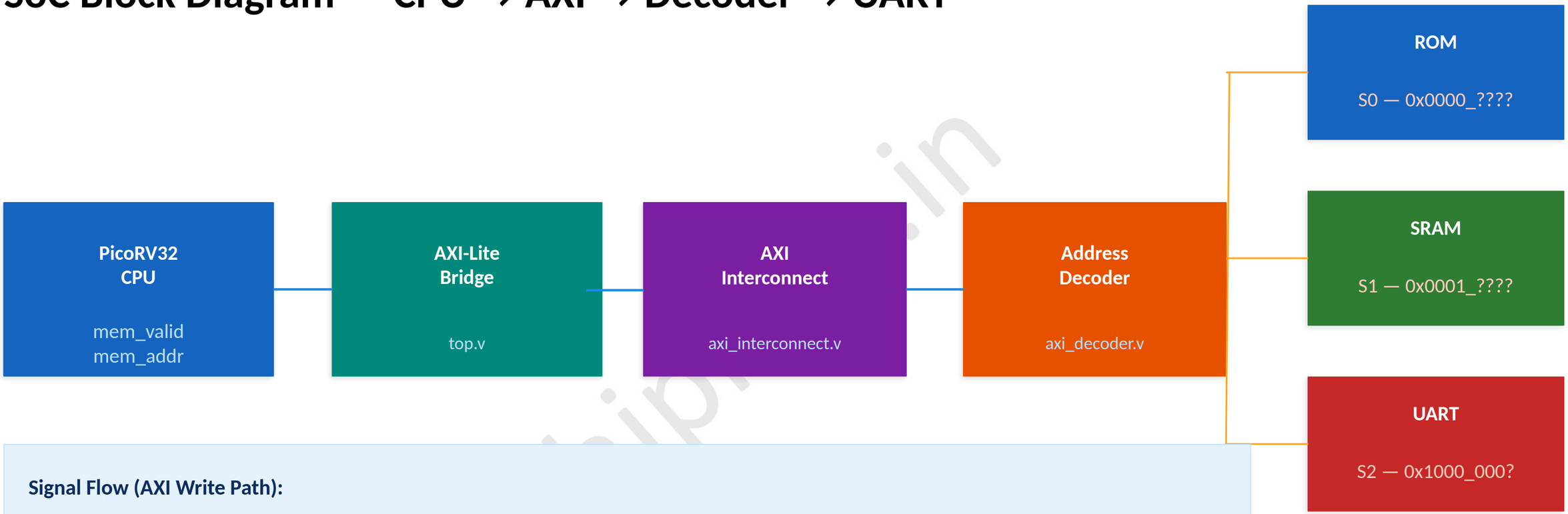
axi_decoder.v — casez match

```
32'h0000_????: sel = 3'b001; // ROM
32'h0001_????: sel = 3'b010; // SRAM
32'h1000_000?: sel = 3'b100; // UART
default:       sel = 3'b001; // safe
```

AXI Address Decode — Project Addresses

Key: Address bits [31:16] select the slave. `casez` wildcard '?' masks lower nibbles. AXI backpressure: AWREADY/WREADY go LOW when `buf_valid=1` (UART busy).

SoC Block Diagram — CPU → AXI → Decoder → UART



Signal Flow (AXI Write Path):

① CPU issues mem_valid + mem_addr = 0x1000_0000 on native bus

② AXI bridge translates to m_axi_awaddr + awvalid handshake

③ Interconnect fans out awvalid to all slaves simultaneously

④ Decoder sets wr_sel = 3'b100 → only uart_axi sees awvalid=1

MMIO in C — UART Driver (fw/main.c)

fw/main.c

```c
/* UART register macros */
#define UART_TX     (*(volatile char*)0x10000000)
#define UART_RX     (*(volatile char*)0x10000004)
#define UART_STATUS (*(volatile int*) 0x10000008)

void uart_putc(char c) {
    UART_TX = c;  /* AXI backpressure stalls CPU */
}

char uart_getc(void) {
    while (!(UART_STATUS & 0x2));
    return UART_RX;  /* clears rx_buf_valid */
}

void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}
```

volatile

Prevents compiler from caching the hardware register read/write — MANDATORY for MMIO

uart_putc

Single store instruction to 0x1000_0000. AXI backpressure stalls CPU if buf_valid=1

uart_getc

Polling loop on bit[1]. When STATUS & 0x2 — RX byte is ready. Reading UART_RX clears flag

uart_puts

Iterates through null-terminated string, calling uart_putc for each character

MMIO Write Path — uart_putc('H') Step-by-Step

① C Statement

```
UART_TX = 'H';   →  compiler: lui a0,0x10000 |
li a1,0x48 | sb a1,0(a0)
```

② PicoRV32 Native Bus

```
mem_valid=1  mem_addr=0x10000000  mem_wstrb=4'h1
 mem_wdata=0x00000048
```

③ CPU→AXI Bridge (top.v)

```
m_axi_awaddr=0x10000000  m_axi_wdata=0x48
m_axi_wstrb=4'h1  awvalid=1
```

④ AXI Interconnect + Decoder

```
casez: 32'h1000_000? → wr_sel=3'b100   Only
uart_axi receives awvalid=1
```

⑤ uart_axi.v

```
w_byte ← 0x48  buf_valid ← 1   AWREADY/WREADY go
LOW (backpressure active)
```

⑥ uart_tx.v FSM

```
IDLE → START → DATA(×8) → STOP   Serial bits on
tx_out pin at configured baud
```

Lab — make soc_pico_uart_mem (MMIO SoC Subsystem)

```
$ make soc_pico_uart_mem
```

Architecture (Sub-module 4)

PicoRV32 CPU → direct native bus → UART peripheral

Simpler than full AXI SoC — pure MMIO concept demo

No AXI bridge or interconnect in this variant

Build steps:

① cd fw && make → rom.hex
② Verilator: picorv32_uart_mem/rtl/ + tb/tb_mem_soc.v
③ ROM loaded from +romhex=rtl/rom.hex
④ Simulation → tb_mem_soc.vcd (142 MB)

Tip: vcd2fst for faster GTKWave loading

Expected Output + GTKWave Signals

```
[TB] RX byte: H
[TB] RX byte: e  ...  (10 greetings)
[TB] FIFO flushed — sending: PING
[TB] RX byte: P  I  N  G
TEST PASSED

GTKWave key signals:
cpu_mem_addr  → 0x1000_0000 (TX writes)
cpu_mem_wstrb → 4'h1 (SB byte store)
cpu_mem_wdata → 0x48 = 'H'
uart_tx       → serial bitstream
```

Design Rules — MMIO Best Practices

V

volatile is Mandatory

Prevents compiler from caching MMIO register reads/writes. Without it, CPU may never re-read STATUS register inside while loop — the C optimizer will hoist it out of the loop.

RT

ASIC-Clean RTL Rule

No initial blocks in RTL. All state must be reset-driven (posedge clk, negedge rst_n). initial blocks are simulation-only; they are ignored by synthesis tools for ASIC.

1T

One Outstanding Transaction

PicoRV32 is single-issue non-pipelined. Only one mem_valid can be outstanding at any time. No out-of-order or speculative memory access. Keeps AXI handshake straightforward.

AL

Address Alignment

AXI-Lite requires word-aligned addresses — bits[1:0] must be 00. Unaligned accesses cause undefined behavior. UART_STATUS at 0x1000_0008 (divisible by 4) is compliant.

AX

AXI Backpressure Rule

AWREADY/WREADY go LOW when uart_axi.buf_valid=1. CPU bridge stalls — no data loss without a FIFO. No explicit wait() needed in firmware; hardware handles flow control.

Sub-module 4 – Key Takeaways

MMIO Core Principle:

CPU uses standard load/store to talk to hardware — no special I/O instructions in RISC-V.

Register Map:

0x1000_0000 → UART TX | 0x1000_0004 → UART RX | 0x1000_0008 → STATUS (bit[1]=rx_buf_valid)

volatile is Non-Negotiable:

Prevents compiler from optimizing away hardware register accesses — required for all MMIO pointers.

Firmware Polling:

while (!(UART_STATUS & 0x2)); — polls STATUS register until RX byte is available.

Lab Command:

make soc_pico_uart_mem → direct native bus SoC (no AXI) | make soc → full AXI-Lite SoC.